GYM MANAGEMENT SYSTEM

Niyog Uprety

Abstract

Gyms manage many daily tasks. Staff must keep track of members, trainers, membership plans, classes and so on. Manual methods create confusion and make it hard to maintain accurate records. The purpose of this project is to design a structured database that supports these operations in a reliable way. The system will store key information, provide fast access to records and help staff carry out updates when needed. It will also improve tracking of membership status, trainer assignments, class attendance etc. This database design creates a clean foundation that a larger gym management system can build upon in the future.

Mission Statement

My mission is to build a database that supports the core operations of a gym. The system will organize information on members, trainers, classes and membership plans in a clear and consistent format. It will allow staff to search, insert, update and delete records without confusion. The goal is to give the gym a dependable structure for managing daily tasks and future expansion.

Mission Objectives

To maintain (enter, update, delete) data on members.
To maintain (enter, update, delete) data on trainers.
To maintain (enter, update, delete) data on membership plans.
To maintain (enter, update, delete) data on workout classes.
To maintain (enter, update, delete) data on enrollments.

To perform searches on members.
To perform searches on trainers.
To perform searches on membership plans.
To perform searches on classes.
To perform searches on enrollments.

To track membership start dates and expiration dates.

To track trainer assignments.

To track class schedules and attendance.

To report on members.

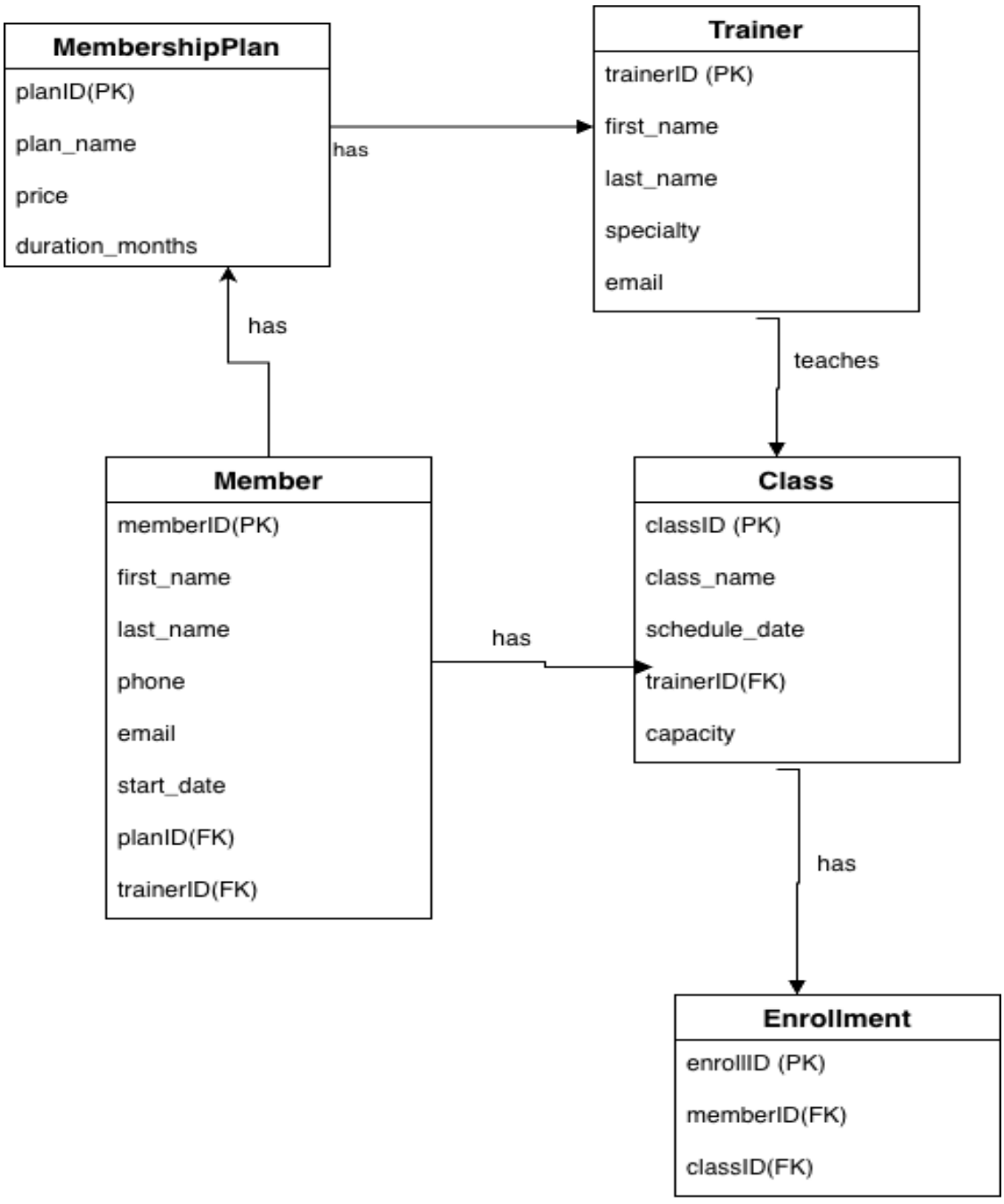
To report on trainers.

To report on plans.

To report on classes.

To report on enrollments.

ER Diagram


MembershipPlan
planID(PK)
plan_name
price
duration_months
has
Trainer
trainerID (PK)
first_name
last_name
specialty
email
has
teaches
Member
memberID(PK)
first_name
last_name
phone
email
start_date
planID(FK)
trainerID(FK)
has
Class
classID (PK)
class_name
schedule_date
trainerID(FK)
capacity
has
Enrollment
enrollID (PK)
memberID(FK)
classID(FK)

Relational Model

MembershipPlan(planID, plan_name, price, duration_months)

MembershipPlan
planID
plan_name
price
duration_months

Trainer(trainerID, first_name, last_name, specialty, email)

Trainer
trainerID
first_name
last_name
specialty
email

Member(memberID, first_name, last_name, phone, email, start_date, planID, trainerID)

Member
memberID
first_name
last_name
phone
email
start_date
planID
trainerID

Class(classID, class_name, schedule_date, trainerID, capacity)

Class
classID
class_name
schedule_date
trainerID
capacity

Enrollment(enrollID, memberID, classID)

Enrollment
enrollID
memberID
classID

Process of Normalization

MembershipPlan(planID, plan_name, price, duration_months)

contains the primary key planID which all the other attributes of the schema functionally depend on. It is in 3NF because there are no transitive dependencies; each attribute is determined only by the primary key. There are no foreign keys in this relation.

Trainer(trainerID, first_name, last_name, specialty, email)

contains the primary key trainerID which all the other attributes of the schema functionally depend on. It is in 3NF because there are no transitive dependencies; each attribute is determined only by the primary key. There are no foreign keys in this relation.

Member(memberID, first_name, last_name, phone, email, start_date, planID, trainerID)

contains the primary key memberID which all the other attributes of the schema functionally depend on. It is in 3NF because there are no transitive dependencies; each attribute is determined only by the primary key. The foreign keys, planID and trainerID, reference the MembershipPlan and Trainer entities in order to describe which plan the member is enrolled in and which trainer they are assigned to.

Class(classID, class_name, schedule_date, trainerID, capacity)

contains the primary key classID which all the other attributes of the schema functionally depend on. It is in 3NF because there are no transitive dependencies; each attribute is determined only by the primary key. The foreign key, trainerID, references the Trainer entity in order to describe which trainer teaches the class.

Enrollment(enrollID, memberID, classID)

contains the primary key enrollID which all the other attributes of the schema functionally depend on. It is in 3NF because there are no transitive dependencies; each attribute is determined only by the primary key. The foreign keys, memberID and classID, reference the Member and Class entities in order to describe which member is enrolled in which class.

Use Case List

Use case: Insert new member
Actor: Staff
Steps:

1. Staff member clicks the “New Member” button.
2. A Member Info window opens.
3. Staff enters first name, last name, phone, email, start date, planID, and trainerID.
4. Program asks for confirmation to create the new member.
5. Staff clicks “Confirm”.

Use case: Delete member
Actor: Staff
Steps:

1. Staff opens the Member Info window for a specific member.
2. Staff clicks the “Delete Member” button.

3. Program asks for confirmation to delete the member.
4. Staff clicks “Confirm”.

Use case: Update member information

Actor: Staff

Steps:

1. Staff opens the Member Info window for a specific member.
2. Staff clicks the “Edit Member” button.
3. The fields become editable.
4. Staff makes the necessary changes.
5. Program asks for confirmation to update the member.
6. Staff clicks “Confirm”.

Use case: Search members

Actor: Staff

Steps:

1. Staff opens the Search window.
2. Staff enters one or more filters such as name, planID, or trainerID.
3. Program displays all members that match the criteria.
4. Staff may double-click a record to open the Member Info window.

Use case: Insert new trainer

Actor: Admin

Steps:

1. Admin clicks the “New Trainer” button.
2. A Trainer Info window opens.
3. Admin enters the trainer’s first name, last name, specialty, and email.
4. Program asks for confirmation to create the trainer.
5. Admin clicks “Confirm”.

Use case: Delete trainer

Actor: Admin

Steps:

1. Admin opens the Trainer Info window for a specific trainer.
2. Admin clicks the “Delete Trainer” button.
3. Program asks for confirmation to delete the trainer.
4. Admin clicks “Confirm”.

Use case: Insert new class

Actor: Admin

Steps:

1. Admin clicks the “New Class” button.
2. A Class Info window opens.
3. Admin enters class name, schedule date, capacity, and trainerID.
4. Program asks for confirmation to create the class.
5. Admin clicks “Confirm”.

Use case: Register member into class

Actor: Staff

Steps:

1. Staff opens the Enrollment window.
2. Staff selects a member from the list.
3. Staff selects a class from the list.
4. Program asks for confirmation to add the enrollment.
5. Staff clicks “Confirm”.

Use case: Search classes

Actor: Staff

Steps:

1. Staff opens the Class Search window.
2. Staff enters filters such as date or trainerID.
3. Program displays all matching classes.
4. Staff may open a Class Info window by selecting a result.

Use Case Realization

1. INSERT USE CASES

Use case: Insert new member

#Insert a new member

```
INSERT INTO Member (first_name, last_name, phone, email, start_date, planID, trainerID)
VALUES ('Fool', 'Hardy', '111-222', 'fool.hardy@gmail.com', '2025-10-11', 1, 1);
```

Use case: Insert new trainer

#Insert a new trainer

```
INSERT INTO Trainer (first_name, last_name, specialty, email, salary)
VALUES ('Tom', 'Hardy', 'Strength', 'tom.hardy@gym.com', 4000);
```

Use case: Insert new class

#Insert a new class

```
INSERT INTO Class (class_name, schedule_date, trainerID, capacity)
VALUES ('Pilates', '2025-06-07', 1, 18);
```

Use case: Insert new membership plan

#Insert a new membership plan

```
INSERT INTO MembershipPlan (plan_name, price, duration_months)
VALUES ('Elite', 120.00, 1);
```

Use case: Register member into class

#Enroll a member into a class

```
INSERT INTO Enrollment (memberID, classID)
VALUES (1, 1);
```

2. UPDATE USE CASES

Use case: Update member information

#Update a member's plan

```
UPDATE Member
SET planID = 2
WHERE memberID = 1;
```

Use case: Update trainer information

#Update trainer salary

```
UPDATE Trainer
SET salary = 5000
WHERE trainerID = 1;
```

Use case: Update membership plan

#Update membership plan price

```
UPDATE MembershipPlan
SET price = 90.00
WHERE planID = 3;
```

Use case: Update class information

```
#Update class capacity

UPDATE Class
SET capacity = 30
WHERE classID = 1;
```

Use case: Update enrollment

```
#Update enrollment (change class for a member)

UPDATE Enrollment
SET classID = 2
WHERE enrollID = 1;
```

3. DELETE USE CASE

Use case: Delete member

```
#Delete a member

DELETE FROM Member
WHERE memberID = 2;
```

Use case: Delete trainer

```
#Delete a trainer

DELETE FROM Trainer
WHERE trainerID = 2;
```

Use case: Delete membership plan

#Delete a membership plan

```
DELETE FROM MembershipPlan
WHERE planID = 3;
```

Use case: Delete class

#Delete a class

```
DELETE FROM Class
WHERE classID = 2;
```

Use case: Delete enrollment

#Delete an enrollment

```
DELETE FROM Enrollment
WHERE enrollID = 2;
```

4. AGGREGATE USE CASES

Use case: Count total members

#Count total members

```
SELECT COUNT(*) AS total_members
FROM Member;
```

Use case: Average membership plan price

#Average membership plan price

```
SELECT AVG(price) AS avg_plan_price
FROM MembershipPlan;
```

Use case: Average trainer salary

#Average trainer salary

```
SELECT AVG(salary) AS avg_salary
FROM Trainer;
```

Use case: Total class capacity

#Total class capacity

```
SELECT SUM(capacity) AS total_capacity
FROM Class;
```

Use case: Count enrollments

#Count all enrollments

```
SELECT COUNT(*) AS total_enrollments
FROM Enrollment;
```

5. JOIN USE CASES

Use case: Member and MembershipPlan

#Show members and the plans they belong to

```
SELECT Member.first_name, Member.last_name, MembershipPlan.plan_name
FROM Member
JOIN MembershipPlan ON Member.planID = MembershipPlan.planID;
```

Use case: Member and Trainer

#Show members and their assigned trainers

```
SELECT Member.first_name, Member.last_name, Trainer.first_name AS trainer_first
FROM Member
JOIN Trainer ON Member.trainerID = Trainer.trainerID;
```

Use case: Class and Trainer

```
#Show classes with the trainer who teaches them

SELECT Class.class_name, Trainer.first_name, Trainer.last_name
FROM Class
JOIN Trainer ON Class.trainerID = Trainer.trainerID;
```

Use case: Member, Enrollment, and Class

```
#Show member enrollments with class names

SELECT Member.first_name, Member.last_name, Class.class_name
FROM Enrollment
JOIN Member ON Enrollment.memberID = Member.memberID
JOIN Class ON Enrollment.classID = Class.classID;
```

6. SEARCH USE CASE

Use case: Search members by plan

```
SELECT * FROM Member
WHERE planID = 1;
```

Testing

Use case: Insert new member

Input data:

INSERT INTO Member (first_name, last_name, phone, email, start_date, planID, trainerID)

VALUES ('Fool', 'Hardy', '111-222', 'fool.hardy@gmail.com', '2025-10-11', 1, 1);

Expected output:

A new entry for the member is added to the Member table.

Actual output:

Query functioned as expected.



me...	first_name	last_name	phone	email	start_date	planID	trainerID
2	Jordan	Lee	333-444	jordan@gmail.com	2024-02-10	2	2
3	Fool	Hardy	111-222	fool.hardy@gmail.com	2025-10-11	1	1
4	Fool	Hardy	111-222	fool.hardy@gmail.com	2025-10-11	1	1

Use case: Update trainer salary

Input data:

UPDATE Trainer SET salary = 5000 WHERE trainerID = 1;

Expected output:

The salary value for trainer 1 updates to 5000.

Actual output:

Query functioned as expected.



trainerID	first_name	last_name	specialty	email	salary
1	John	Cruz	Strength	john@gym.com	5000.00
2	Maria	Lopez	Cardio	maria@gym.com	4200.00
NULL	NULL	NULL	NULL	NULL	NULL

Use case: **Delete an enrollment**

Input data:

DELETE FROM Enrollment WHERE enrollID = 2;

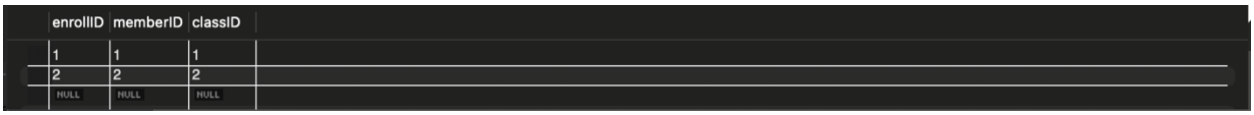
Expected output:

The enrollment entry with ID 2 is removed from the Enrollment table.

Actual output:

Query functioned as expected.

Before:



After:



Use case: **Count total members**

Input data:

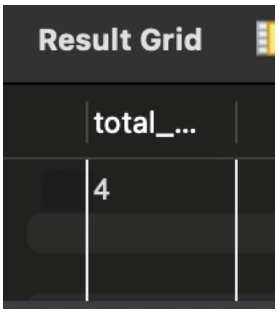
SELECT COUNT(*) AS total_members FROM Member;

Expected output:

A single number appears showing how many members are stored in the Member table.

Actual output:

Query functioned as expected.



Use case: Show members and their membership plans

Input data:

SELECT Member.first_name, Member.last_name, MembershipPlan.plan_name

FROM Member

JOIN MembershipPlan ON Member.planID = MembershipPlan.planID;

Expected output:

A result table appears listing members along with their corresponding membership plan.

Actual output:

Query functioned as expected.



first_name	last_name	plan_name
Alex	Kim	Basic
Fool	Hardy	Basic
Fool	Hardy	Basic
Jordan	Lee	Standard

CONCLUSION

The Gym Membership Management System provides a simple structure for storing and managing core information of the business. The database design keeps the relationships clear and helps organize data in a way that is easy to work with. The SQL statements show how each part of the system performs tasks like adding new records, updating details, removing entries, running summary checks and combining information through joins. All tests worked as expected, and the results confirmed that the database responds correctly to each operation. This project gives a clear view of how a small system can support basic gym operations using a relational database.

Reference

- Draw.io Online Diagram Tool
  https://app.diagrams.net/
- W3Schools SQL Reference (Used for confirming basic SQL functions)
  https://www.w3schools.com/sql/
- Example Report: Animal Shelter Database